

DPDP COMPLIANCE AUDIT

ProcessPilot-AI

Digital Personal Data Protection Act 2023



Grade D — At Risk

Scanned: 14 April 2026

Files indexed: 76

Languages: express, mongoose, passport, dotenv, cors, cookie-parser

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

Quick Actions for Developers

Fix these first to improve your compliance score:

- **CONSENT_WITHDRAWAL_MISSING** (codebase-wide)
- **INCOMPLETE_DELETION_COVERAGE** (codebase-wide)
- **GRIEVANCE_OFFICER_MISSING** (codebase-wide)
- **PASSWORD_NOT_HASHED** → backend/models/user.model.js
- **AUDIT_TRAIL_PARTIAL** (codebase-wide)

This codebase has 6 critical compliance gap(s) that require immediate attention under the Digital Personal Data Protection Act 2023. An additional 9 significant gap(s) were identified that should be addressed within 30 days. The estimated total remediation effort is 4.4–16.1 engineering days.

The application collects user email, password, name, and Google OAuth tokens. The biggest DPDP risk is the potential for unauthorized access to user credentials and PII stored in the User model, especially given the use of JWTs and cookie-based authentication.

Total findings: 20

- Rule Engine: 16
- AI Deep Review: 4

By Severity:

- HIGH: 6
- MEDIUM: 9
- LOW: 2
- PASS: 3

Developer Priority:

- Fix Now: 5
- Fix This Sprint: 3

Tech Stack: express, mongoose, passport, dotenv, cors

Risk Profile: The application collects user email, password, name, and Google OAuth tokens. The biggest DPDP risk is the potential for unauthorized access to user credentials and PII stored in the User model, especially given the use of JWTs and cookie-based authentication.

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	22/22 (100%)	PASS
Section 6(6)	8	0/8 (0%)	FAIL
Section 7	4	4/4 (100%)	PASS
Section 8(1)	18	0/18 (0%)	FAIL
Section 8	10	2.0/10 (20%)	FAIL
Section 8(3)	8	0/8 (0%)	FAIL
Section 8(4)	8	4.0/8 (50%)	WARN
Section 8(6)	8	6.8/8 (85%)	PASS

Section 9	6	6/6 (100%)	PASS
Section 11	6	6/6 (100%)	PASS
Section 16	4	2.0/4 (50%)	WARN
Section 5	2	0/2 (0%)	FAIL
Section 13 — Grievance Redressal	4	0/4 (0%)	FAIL

Estimated Total Remediation Effort: 4.4–16.1 engineering days

Scan Coverage

Files analyzed: 76

Rule	Section	Severity	File
CONSENT_WITHDRAWAL_MISSING	Section 6(6) — Consent Withdrawal	HIGH	N/A
INCOMPLETE_DELETION_COVERAGE	Section 8 — Data Principal Rights	HIGH	N/A
GRIEVANCE_OFFICER_MISSING	Section 13 — Grievance Redressal	HIGH	N/A
PLAINTEXT_PII_IN_LOGS	Section 8(1) — Security	HIGH	MULTIPLE
PASSWORD_NOT_HASHED	Section 8(1) — Security	MEDIUM	backend/models/user.model.js
AUDIT_TRAIL_PARTIAL	Section 8(4) — Audit Trail	MEDIUM	N/A
RETENTION_PARTIAL	Section 8(3) — Retention	MEDIUM	N/A
CROSS_BORDER_TRANSFER_RISK	Section 16 — Cross-Border Transfer	MEDIUM	backend/controllers/google.controller.js
THIRD_PARTY_PII_SHARING	Section 5 — Notice	MEDIUM	backend/services/testEmail.js
RETENTION_MISSING	Section 8(3) — Retention	MEDIUM	backend/models/group.model.js
RETENTION_MISSING	Section 8(3) — Retention	MEDIUM	backend/models/workflow.model.js
NO_BREACH_ALERTING	Section 8(6) — Breach	LOW	N/A
DATA_PORTABILITY_MISSING	Section 11 — Data Portability	LOW	N/A
CONSENT_PRESENT	Section 6 — Consent	PASS	backend/routes/google.route.js
DELETION_MECHANISM_PRESENT	Section 8 — Data Principal Rights	PASS	backend/routes/auth.route.js
DATA_ACCESS_PRESENT	Section 11 — Right to Access	PASS	frontend/src/pages/Anyanalytics.tsx
DEEP_REVIEW_VALIDATED	Section 32 — Security of processing	HIGH	backend/config/google.js

DEEP_REVIEW_VALIDATED	Section 5	HIGH	backend/routes/group.routes.js
DEEP_REVIEW_VALIDATED	Section 5	MEDIUM	backend/routes/prompt.route.js
DEEP_REVIEW_VALIDATED	Section 5	MEDIUM	backend/models/group.model.js

1 finding(s) show rule-engine fallback text only (LLM enrichment cap or parse failure).

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (12–46 hours)

Fix immediately — these represent the highest legal exposure.

- Consent Withdrawal Missing (1–4 hrs)
- Incomplete Deletion Coverage (4–16 hrs)
- Grievance Officer Missing (1–4 hrs)
- Plaintext Pii In Logs — MULTIPLE (2–6 hrs)
- Deep Review Validated — config/google.js (2–8 hrs)
- Deep Review Validated — routes/group.routes.js (2–8 hrs)

Week 2 — High Priority (19–69 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- Password Not Hashed — models/user.model.js (1–3 hrs)
- Audit Trail Partial (2–8 hrs)
- Retention Partial (2–8 hrs)
- Cross Border Transfer Risk — controllers/google.controller.js (4–12 hrs)
- Third Party Pii Sharing — services/testEmail.js (2–6 hrs)
- Retention Missing — models/group.model.js (2–8 hrs)
- Retention Missing — models/workflow.model.js (2–8 hrs)
- Deep Review Validated — routes/prompt.route.js (2–8 hrs)
- Deep Review Validated — models/group.model.js (2–8 hrs)

Week 4 — Advisory (4–14 hours)

Address before next compliance review.

- No Breach Alerting (2–6 hrs)
- Data Portability Missing (2–8 hrs)

Total estimated remediation effort: 35–129 hours (4.4–16.1 engineering days)

Details

Finding 1 of 20 — ■ FIX NOW

[Section 6(6) — Consent Withdrawal]

1. [HIGH] CONSENT_WITHDRAWAL_MISSING — N / A**Estimated fix time:** 1–4 hours

Consent is collected but no withdrawal mechanism detected. DPDP Section 6(6) requires consent withdrawal to be as easy as giving consent. Users must be able to revoke consent at any time.

Risk:

The application lacks a comprehensive data deletion mechanism, meaning user data may persist in the system even after a deletion request. This violates DPDP Section 8, which mandates the complete erasure of personal data upon request, potentially leading to continued unauthorized access or misuse of residual data.

Fix:

1. Implement a 'delete account' feature in the user profile section.
2. Create a new controller function `deleteUserAccount` that handles the deletion of user data from the `User` model and any related models (e.g., `activity\_model`, `workflow\_model`).
3. Ensure that all references to the user's PII are removed or anonymized across all data stores, including logs and any associated third-party services.
4. Add a new API endpoint, e.g., `DELETE /api/users/me`, that triggers the `deleteUserAccount` function after proper authentication and authorization.

DPDP:

Section 8 — Data Principal Rights: This section mandates that data principals have the right to request the erasure of their personal data.

Example:

```
import { User } from "../models/user.model.js"; import { Activity } from
"../models/activity.model.js"; // Assuming this model exists export const deleteUserAccount =
async (req, res) => { try { const userId = req.user.id; // Assuming user ID is available from
auth middleware await User.findByIdAndDelete(userId); await Activity.deleteMany({ userId:
userId }); // Example for related data // Add more deletions for other related models as needed
res.status(200).json({ success: true, message: "Account deleted successfully" }); } catch
(error) { res.status(500).json({ success: false, message: "Failed to delete account" }); } };
```

Finding 2 of 20 — ■ FIX NOW

[Section 8 — Data Principal Rights]

2. [HIGH] INCOMPLETE_DELETION_COVERAGE — N / A:15**Evidence snippet:** The route `router.delete("/:id", verifyToken, deleteUser);` indicates a user deletion endpoint.**Estimated fix time:** 4–16 hours

Deletion mechanism detected but may not cover all PII storage. Coverage: 0% of inferred PII entities (0/5 including FK dependents). Uncovered: activity_model, connectdb, group_model, user_model, workflow_model. DPDP Section 8 requires complete erasure across all systems.

Entity deletion coverage: 0% (0 covered / 5 uncovered)**Uncovered entities:** activity_model, connectdb, group_model, user_model, workflow_model**Entity→file hints:** activity_model: models/activity.model.js; connectdb: db/connectDB.js; group_model: models/group.model.js; user_model: models/user.model.js; workflow_model: models/workflow.model.js**Risk:**

The application has endpoints for deleting users and groups, but the rule engine indicates that not all PII entities are covered by deletion. This means that personal data might remain in other parts of the system, violating the DPDP's requirement for complete data erasure and potentially exposing sensitive information.

Fix:

1. Identify all data models that store or reference personal information (PII) beyond the `User` model, such as `activity\_model`, `workflow\_model`, etc.
2. For each identified model, implement corresponding deletion logic within the relevant controller or a dedicated service.
3. Update the `deleteUser` controller in `auth.controller.js` to cascade deletions or remove associated data from these other models.
4. Ensure that any foreign key constraints in the database are configured to handle deletions appropriately (e.g., `ON DELETE CASCADE` if applicable and safe).

DPDP:

Section 8 — Data Principal Rights: This section mandates that data principals have the right to request the erasure of their personal data.

Example:

```
import { User } from "../models/user.model.js"; import { Workflow } from
"../models/workflow.model.js"; // Example: Assuming workflow model stores user data export
const deleteUser = async (req, res) => { try { const userId = req.params.id; const user = await
User.findByIdAndDelete(userId); if (!user) { return res.status(404).json({ message: "User not
found" }); } // Delete associated workflows await Workflow.deleteMany({ userId: userId });
res.status(200).json({ message: "User and associated data deleted successfully" }); } catch
(error) { res.status(500).json({ message: "Server error", error: error.message }); } };
```

Finding 3 of 20 — ■ FIX NOW

[Section 13 — Grievance Redressal]

3. [HIGH] GRIEVANCE_OFFICER_MISSING — N / A**Estimated fix time:** 1–4 hours

No compliant Grievance Officer / DPO publication found. A PASS requires grievance-related text or a dedicated route ****and**** a contact email or dedicated grievance/DPO URL. Tiny repos without a grievance route are not marked PASS.

Risk:

Personal Identifiable Information (PII) such as email addresses is being logged in plaintext, which poses a security risk. If these logs are compromised or accessed by unauthorized personnel, sensitive user data could be exposed, violating DPDP Section 8(1) regarding data security.

Fix:

1. Remove or redact sensitive PII fields (like email addresses and potentially other user details) from `console.log` statements in `backend/ingest/functions/on-signup.js`.
2. Implement a logging middleware or utility function that automatically masks PII fields before they are written to logs.
3. Review all other logging statements across the application to ensure no other PII is exposed in plaintext.
4. Configure log rotation and access controls to further protect log data.

DPDP:

Section 8(1) — Security: This section requires data controllers to implement appropriate security measures to protect personal data.

Example:

```
import { ingest } from "../client.js"; import { User } from "../models/user.model.js";
import { NonRetriableError } from "ingest"; import { sendMail } from "../utils/mailer.js";
export const onUsersignup = ingest.createFunction( { id: "on-user-signup"}, { event:
"user/signup" }, async ({ event, step }) => { try { const {email} = event.data; //
console.log(email) // Removed or redacted const user = await step.run("get-user-email",
async())=> { const userobject = await User.findOne({ email }) if(!userobject) { throw new
NonRetriableError("User no longer exists") } return userobject; }) // console.log(user) // Log
only non-sensitive fields or redact PII console.log(`User found with ID: ${user._id}`); //
Example of logging non-PII await step.run("send-welcome-email", async () => { const subject =
"Welcome to ProcessPilot" const message = `Hi, welcome to ProcessPilot!\n\n\n Thanks for
signing up. We're glad to have you` sendMail(user.email,subject,message) }) return {success:
true} } catch (error) { console.error("Error running Step", error.message) return {success:
false} } } )
```

Finding 4 of 20 — ■ FIX THIS SPRINT

[Section 8(1) — Security]

4. [HIGH] PLAINTEXT_PII_IN_LOGS — MULTIPLE:90**Evidence snippet:** console.log("Sending email to:", to);**Estimated fix time:** 2–6 hours**Affected files (4):**

- backend/controllers/google.controller.js
- backend/inngest/functions/on-signup.js
- backend/middleware/verifyToken.js
- backend/services/testEmail.js

PII keyword appears in logging call. Detected in 4 files: google.controller.js, on-signup.js, verifyToken.js, testEmail.js

Risk:

Manual review required. Rule 'PLAINTEXT_PII_IN_LOGS' fired on see finding scope — see rule description for details.

Fix:

1. Review the finding description above and apply fixes per your compliance process.

DPDP:

Section 8(1) — Security

Finding 5 of 20 — ■ REVIEW & FIX

[Section 8(1) — Security]

5. [MEDIUM] PASSWORD_NOT_HASHED — backend / models / user.model.js:26**Evidence snippet:** The code uses bcryptjs.hash to hash the password before saving it.**Estimated fix time:** 1–3 hours

Model file has password field but no hash/bcrypt/argon/pbkdf2/scrypt/encrypt reference.

Risk:

The application correctly hashes passwords using bcryptjs before storing them, which is a strong security practice. This finding appears to be a false positive as the evidence shows password hashing is implemented.

Fix:

1. Manual review required — LLM did not provide concrete fix steps.

DPDP:

Section 8(1) — Security

Example:

```
const hashedPassword = await bcryptjs.hash(password, 10); const user = new User({ email,
password: hashedPassword, name, verificationToken, verificationTokenExpiresAt: Date.now() + 24
* 60 * 60 * 1000, // 24 hours }); await user.save();
```


Finding 6 of 20 — ■ REVIEW & FIX

[Section 8(4) — Audit Trail]

6. [MEDIUM] AUDIT_TRAIL_PARTIAL — N / A:19**Evidence snippet:** The `getAllUsers` endpoint is protected by `verifyToken` middleware.**Estimated fix time:** 2–8 hours

Partial audit trail detected. Found: change tracking. Missing: access logging, audit model.

Risk:

The application has implemented JWT-based authentication for many routes, indicating some level of access control. However, there is no explicit logging of user access to these protected resources, nor is there a centralized audit model to record these events, which is a gap in comprehensive auditing.

Fix:

1. Implement an audit logging middleware that captures user ID, timestamp, and accessed endpoint for all requests protected by `verifyToken`.
2. Define a Mongoose schema for audit logs, including fields like `userId`, `action`, `timestamp`, `details`, and `ipAddress`.
3. Create a service or controller function to save these audit logs to the database.
4. Ensure the audit log model and logging mechanism are integrated into the application's request lifecycle, particularly for authenticated routes.

DPDP:

Section 8(4) — Audit Trail

Example:

```
import { User } from "../models/user.model.js"; // Assuming User model is available import {
AuditLog } from "../models/auditLog.model.js"; // Define this model export const
auditLogMiddleware = async (req, res, next) => { if (req.user) { // Check if user is
authenticated via verifyToken try { const newLog = new AuditLog({ userId: req.user._id,
action: `${req.method} ${req.originalUrl}`, ipAddress: req.ip }); await newLog.save(); } catch
(error) { console.error("Audit logging failed:", error); } } next(); };
```

Finding 7 of 20 — ■ REVIEW & FIX

[Section 8(3) — Retention]

7. [MEDIUM] RETENTION_PARTIAL — N / A:36

Evidence snippet: The User model has a `timestamps: true` option, which automatically adds `createdAt` and `updatedAt` fields. This indicates some form of change tracking.

Estimated fix time: 2–8 hours

Retention signals found in 1 model(s) but 2 PII model(s) have no retention policy. Uncovered: group.model.js, workflow.model.js.

Risk:

While the `User` model has basic timestamping for creation and updates, there's no explicit data retention policy defined for user data. This means data might be stored indefinitely, increasing the risk associated with holding onto personal information longer than necessary.

Fix:

1. Define a data retention policy for user data, specifying the duration for which different types of data should be kept.
2. Implement a scheduled task (e.g., using `node-cron` or a similar library) to periodically identify and delete user data that has exceeded its retention period.
3. Consider adding a `deletedAt` or `isArchived` field to models to soft-delete records before permanent deletion, allowing for easier recovery if needed.
4. Document the data retention policy and the implementation details within the project's compliance documentation.

DPDP:

Section 8(3) — Retention

Example:

```
import mongoose from 'mongoose'; import cron from 'node-cron'; const userSchema = new
mongoose.Schema({ // ... other fields createdAt: { type: Date, default: Date.now }, updatedAt:
{ type: Date, default: Date.now }, // Add a field for retention period if needed, e.g.,
expiresAt // expiresAt: { type: Date, default: () => new Date(Date.now() + 365 * 24 * 60 * 60 *
1000) } // Example: 1 year }, { timestamps: true }); // Example cron job to clean up old data
(run this in your app setup) // cron.schedule('0 0 * * *', async () => { //
console.log('Running data retention cleanup...'); // await User.deleteMany({ expiresAt: { $lt:
new Date() } }); // }); export const User = mongoose.model('User', userSchema);
```

Finding 8 of 20 — ■ REVIEW & FIX

[Section 16 — Cross-Border Transfer]

8. [MEDIUM] CROSS_BORDER_TRANSFER_RISK — backend / controllers / google.controller.js:12**Evidence snippet:** "https://www.googleapis.com/auth/gmail.send",**Estimated fix time:** 4–12 hours

Foreign cloud storage/CDN detected with PII models in repo. Cross-border transfer risk.

Risk:

The application integrates with Google services, which involves handling OAuth tokens and potentially accessing user data from Google. If these tokens or PII are transferred to or processed by Google servers located outside of the user's jurisdiction without appropriate safeguards, it constitutes a cross-border data transfer risk.

Fix:

1. Review the specific Google APIs and scopes being used to ensure they are necessary for the application's functionality.
2. Consult Google Cloud's documentation on data residency and cross-border transfer policies to understand where data is processed.
3. Implement data minimization by only requesting the minimum necessary scopes and storing only essential user data and tokens.
4. If direct cross-border transfers are unavoidable, ensure appropriate legal mechanisms (e.g., Standard Contractual Clauses) are in place and documented.

DPDP:

Section 16 — Cross-Border Transfer

Example:

```
import { User } from "../models/user.model.js"; import { google } from 'googleapis'; //
Example of how to use stored tokens to access Google APIs const getGoogleClient = async
(userId) => { const user = await User.findById(userId); if (!user || !user.accessToken) {
throw new Error('User not found or not authenticated with Google'); } const auth = new
google.auth.OAuth2(); auth.setCredentials({ access_token: user.accessToken, refresh_token:
user.refreshToken, expiry_date: user.tokenExpiry ? user.tokenExpiry.getTime() : undefined });
return google.oauth2({ version: 'v2', auth }); };
```

Finding 9 of 20 — ■ REVIEW & FIX

[Section 5 — Notice]

9. [MEDIUM] THIRD_PARTY_PII_SHARING — backend / services / testEmail.js:10**Evidence snippet:** import { emailDraft } from "../utils/ai.js"; // ■ Adjust this path if needed**Estimated fix time:** 2–6 hours

Personal data present in file that imports third-party SDK. Data may be shared without user awareness or notice (DPDP Section 5).

Risk:

Personal data, specifically email addresses, is processed within this file, potentially involving third-party AI utilities. Without clear notice, users may not be aware their data is being used for these purposes, violating DPDP Section 5.

Fix:

1. Add a clear notice to users about the processing of their email for AI-driven email drafting and testing, including details about any third-party services involved.
2. Review the `../utils/ai.js` utility to ensure it handles PII securely and in accordance with DPDP principles.
3. If `../utils/ai.js` is a third-party SDK, ensure its terms of service and privacy policy are reviewed and align with DPDP requirements.
4. Consider implementing a mechanism to obtain user consent before processing their data for AI-driven features.

DPDP:

Section 5 — Notice requires that individuals be informed about the collection, processing, and sharing of their personal data.

Example:

```
// Example of adding a notice in a user-facing component or API response // This is a conceptual example and needs to be integrated into your frontend or relevant API endpoint.
const userNotice = { title: "Data Usage for AI Features", description: "Your email may be used with AI tools to draft and test emails. We ensure data is handled securely and in compliance with privacy regulations.", consentRequired: true // Or false, depending on your implementation }; // In your API response or user profile section, display this notice. // For example, in an Express route: // res.json({ data: ..., notice: userNotice });
```

Finding 10 of 20 — ■ REVIEW & FIX

[Section 8(3) — Retention]

10. [MEDIUM] RETENTION_MISSING — backend / models / group.model.js:26

Evidence snippet: The `groupSchema` includes a `createdBy` field, which is a reference to the `User` model. This links group data to a specific user.

Estimated fix time: 2–8 hours

Model/schema file contains PII fields but no retention or expiry signals detected. Data may be retained indefinitely.

Risk:

The `Group` model stores email addresses, which are PII, without any defined retention policy. This could lead to indefinite storage of personal data, violating DPDP Section 8(3).

Fix:

1. Add a `retentionPeriodDays` field to the `groupSchema` to specify how long group data, including emails, should be stored.
2. Implement a scheduled task (e.g., using a cron job or a background worker) that periodically deletes groups and their associated data after their retention period has expired.
3. Update the `Group` model to include logic for data deletion or anonymization based on the `retentionPeriodDays`.
4. Document the data retention policy for groups in your privacy policy.

DPDP:

Section 8(3) — Data minimization and retention requires that personal data should not be kept for longer than is necessary for the purposes for which it is processed.

Example:

```
import mongoose from "mongoose"; const groupSchema = new mongoose.Schema( { name: { type: String, required: true, unique: true }, description: { type: String, default: "" }, emails: { type: [String], required: true, validate: { validator: (arr) => arr.length > 0, message: "At least one email is required" }, }, createdBy: { type: mongoose.Schema.Types.ObjectId, ref: "User" }, googleGroupId: { type: String, default: null }, retentionPeriodDays: { type: Number, default: 365 } // Example: 1 year retention }, { timestamps: true } ); export const Group = mongoose.model("Group", groupSchema);
```

Finding 11 of 20 — ■ REVIEW & FIX

[Section 8(3) — Retention]

11. [MEDIUM] RETENTION_MISSING — backend / models / workflow.model.js:67

Evidence snippet: The `workflowSchema` includes a `userId` field, which is a reference to the `User` model. This links workflow data to a specific user.

Estimated fix time: 2–8 hours

Model/schema file contains PII fields but no retention or expiry signals detected. Data may be retained indefinitely.

Risk:

The `Workflow` model stores user-related data, including potentially sensitive information within steps and parameters, without a defined retention period. This could lead to indefinite storage of personal data, violating DPDP Section 8(3).

Fix:

1. Add a `retentionPeriodDays` field to the `workflowSchema` to define how long workflow data should be retained.
2. Implement a scheduled job to automatically delete or anonymize workflow data that exceeds its retention period.
3. Review the `steps` and `parameters` within the `workflowSchema` to identify and potentially redact or anonymize PII before applying retention policies.
4. Document the data retention policy for workflows in your privacy policy.

DPDP:

Section 8(3) — Data minimization and retention requires that personal data should not be kept for longer than is necessary for the purposes for which it is processed.

Example:

```
import mongoose from 'mongoose'; // ... (other schema definitions) const workflowSchema = new
mongoose.Schema({ userId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true
}, name: { type: String }, description: { type: String, required: true }, steps: [stepSchema],
status: { type: String, enum: ["idle", "running", "completed", "failed"], default: "idle" },
executionCount: { type: Number, default: 1 }, successRate: { type: Number, default: 0 },
lastRun: { type: Date }, retentionPeriodDays: { type: Number, default: 90 } // Example: 90
days retention }, { timestamps: true }); export const Workflow = mongoose.model("Workflow",
workflowSchema);
```

Finding 12 of 20 — ■ BACKLOG

[Section 8(6) — Breach]

12. [LOW] NO_BREACH_ALERTING — N / A**Estimated fix time:** 2–6 hours

Structured logging detected but no breach notification mechanism found. DPDP Section 8(6) requires notifying affected individuals and the Authority without undue delay when a breach occurs. Logging alone is insufficient — implement automated alerting (Sentry, PagerDuty, or equivalent) with escalation to a compliance team.

Risk:

The application lacks an automated mechanism to alert relevant parties in the event of a data breach. This absence means that timely notification to affected individuals and the Data Protection Authority, as required by DPDP Section 8(6), may not occur, increasing the potential harm from a breach.

Fix:

1. Integrate a third-party service like Sentry, Datadog, or PagerDuty for real-time error and security event monitoring.
2. Configure alerts within the chosen service to trigger notifications (e.g., email, Slack) to a designated compliance team or individual upon detection of suspicious activities or potential breaches.
3. Establish an incident response plan that outlines the steps to be taken upon receiving a breach alert, including investigation, containment, and notification procedures.
4. Ensure the compliance team is trained on the incident response plan and the use of the alerting system.

DPDP:

Section 8(6) — Breach notification requires that data controllers notify affected individuals and the supervisory authority without undue delay when a personal data breach occurs.

Example:

```
// Example using Sentry for error monitoring and alerting // 1. Install Sentry SDK: npm
install @sentry/node // 2. Initialize Sentry in your application's entry point (e.g.,
backend/index.js) import * as Sentry from "@sentry/node"; Sentry.init({ dsn:
process.env.SENTRY_DSN, // Your Sentry DSN from environment variables integrations: [ // Add
integrations here, e.g., for Express new Sentry.Integrations.Http({ tracing: true, }), ],
tracesSampleRate: 1.0, // Adjust as needed }); // In your Express app setup (e.g.,
backend/index.js): // app.use(Sentry.Handlers.requestHandler()); //
app.use(Sentry.Handlers.errorHandler()); // Configure Sentry alerts in your Sentry dashboard
for specific error types or events.
```

Finding 13 of 20 — ■ BACKLOG

[Section 11 — Data Portability]

13. [LOW] DATA_PORTABILITY_MISSING — N / A**Estimated fix time:** 2–8 hours

No data export or portability endpoint detected. DPDP Section 11 requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Risk:

The absence of a data portability endpoint means users cannot easily obtain a copy of their personal data, which is a requirement under DPDP Section 11. This can lead to user dissatisfaction and potential non-compliance penalties.

Fix:

1. Create a new route (e.g., `/api/data-portability`) in `backend/routes/auth.route.js` that is protected by authentication.
2. Implement a controller function (e.g., `exportUserData`) in `backend/controllers/auth.controller.js` to fetch user data.
3. In the controller, query the database for the authenticated user's personal data (email, name, etc.).
4. Format the retrieved data into a machine-readable format (e.g., JSON) and send it as a file download response.

DPDP:

Section 11(1) — Data Portability requires providing personal data in a commonly used, machine-readable format.

Example:

```
import express from 'express'; import { verifyToken } from '../middleware/verifyToken.js';
import { exportUserData } from '../controllers/auth.controller.js'; const router =
express.Router(); router.get('/export-data', verifyToken, exportUserData); export default
router; // In controllers/auth.controller.js: // export const exportUserData = async (req,
res) => { // // Fetch user data based on req.user.id // // Format as JSON and send as file
download // };
```

Finding 14 of 20 — ■ BACKLOG

[Section 6 — Consent]

14. [PASS] CONSENT_PRESENT — backend / routes / google.route.js

Framework auth/consent middleware detected in this file.

Finding 15 of 20 — ■ BACKLOG

[Section 8 — Data Principal Rights]

15. [PASS] DELETION_MECHANISM_PRESENT — backend / routes / auth.route.js

Data deletion mechanism detected. Found 3 deletion endpoint(s) and 12 deletion signal(s).

Note: Verify deletion cascades to all PII tables (sessions, logs, related records).

Fix:

1. Verify deletion cascades to: sessions, audit_logs, third-party sync records.
2. Use database CASCADE DELETE or explicit multi-table deletion in deletion service.
3. Test deletion completeness: create account, delete, query all PII tables for orphaned data.

Finding 16 of 20 — ■ BACKLOG

[Section 11 — Right to Access]

16. [PASS] DATA_ACCESS_PRESENT — frontend / src / pages / Anyalytics.tsx

Data access endpoint detected for authenticated users.

Finding 17 of 20 — ■ FIX NOW

[Section 32 — Security of processing]

17. [HIGH] DEEP_REVIEW_VALIDATED [AI Deep Review] — backend / config / google.js

Estimated fix time: 2–8 hours

The Google OAuth client secret is loaded directly from
`process.env.GOOGLE\_CLIENT\_SECRET`.

Finding 18 of 20 — ■ FIX NOW

[Section 5]

18. [HIGH] DEEP_REVIEW_VALIDATED [AI Deep Review] — backend / routes / group.routes.js

Estimated fix time: 2–8 hours

The routes for group management (create, get, update, delete, add/remove member) are not consistently protected by the verifyToken middleware.

Finding 19 of 20 — ■ FIX THIS SPRINT

[Section 5]

19. [MEDIUM] DEEP_REVIEW_VALIDATED [AI Deep Review] — backend / routes / prompt.route.js

Estimated fix time: 2–8 hours

The POST route '/evaluate' for prompt evaluation is not protected by the `verifyToken` middleware.

Finding 20 of 20 — ■ FIX THIS SPRINT

[Section 5]

20. [MEDIUM] DEEP_REVIEW_VALIDATED [AI Deep Review] — backend / models / group.model.js

Estimated fix time: 2–8 hours

The 'emails' field in the Group model stores an array of strings, which are likely email addresses (PII). There is no indication of encryption or masking for this data.

AI Deep Compliance Review

This section shows AI analysis that identified compliance gaps beyond deterministic rules. Validated findings are integrated into the main findings list above with [AI Deep Review] labels.

Overall Risk: CRITICAL

The application exhibits critical vulnerabilities in its configuration and API layers, directly exposing sensitive user PII and credentials. Inconsistent application of authentication middleware across critical endpoints, coupled with the direct exposure of Google OAuth secrets, creates a high risk of unauthorized access and data breaches. Furthermore, the lack of explicit consent mechanisms for data collection and processing, particularly within AI-driven workflows, poses significant DPDP compliance challenges.

Estimated remediation: 15 engineering day(s)

Most Critical Action: Implement a secure secret management solution (e.g., HashiCorp Vault, AWS Secrets Manager) to store and retrieve the Google OAuth client secret, removing it from environment variables.

- **Inconsistent or missing authentication and authorization controls** [HIGH] — Article 6 (Lawfulness of processing), Article 32 (Security of processing) — layers: Authentication & Session, API Routes & Controllers
- **Insufficient PII protection and consent management** [HIGH] — Article 5 (Principles relating to processing of personal data), Article 7 (Conditions for consent) — layers: Authentication & Session, Data Models & Storage, Third-Party & PII Flows
- **Exposure of sensitive credentials and secrets** [HIGH] — Article 32 (Security of processing) — layers: Configuration & Secrets, API Routes & Controllers

1. [HIGH] Google OAuth Client Secret exposed via environment variable — Configuration & Secrets — Article 32 (Security of processing)

The Google OAuth client secret is loaded directly from `process.env.GOOGLE\_CLIENT\_ID` and `process.env.GOOGLE\_CLIENT\_SECRET`.

2. [HIGH] Missing verifyToken middleware on group routes — API Routes & Controllers — Article 6 (Lawfulness of processing), Article 32 (Security of processing)

The routes for group management (create, get, update, delete, add/remove member) are not consistently protected by the verifyToken middleware.

3. [MEDIUM] PII in group emails not encrypted or masked — Data Models & Storage — Article 5 (Principles relating to processing of personal data)

The 'emails' field in the Group model stores an array of strings, which are likely email addresses (PII). There is no indication of encryption or masking for this data.

4. [MEDIUM] PII in workflow parameters not encrypted or masked — Data Models & Storage — Article 5 (Principles relating to processing of personal data)

The 'parameters' field within the 'steps' schema is of type 'Mixed' and can dynamically hold various schemas, including potentially sensitive information like email addresses in 'emailParamsSchema'.

5. [LOW] Email sent without explicit user consent — Authentication & Session — Article 7 (Conditions for consent)

The 'send-welcome-email' step sends an email to the user upon signup without an explicit consent mechanism being visible in this code.

Layer Breakdown

Configuration & Secrets — 1 finding(s). The configuration layer primarily relies on environment variables for sensitive information like Google OAuth credentials. While common, this practice introduces a significant risk if these environment variables are not managed with robust security controls, potentially exposing secrets.

Authentication & Session — 4 finding(s). The authentication and session layer correctly uses JWT for session management and bcrypt for password hashing. However, there are opportunities to improve data protection by ensuring explicit consent for communications and by refining logging practices in production environments.

API Routes & Controllers — 4 finding(s). The API routes layer shows some inconsistencies in applying authentication middleware, particularly on group management and prompt evaluation endpoints. Configuration loading also appears to be handled in a non-standard way.

Data Models & Storage — 3 finding(s). The data models generally define fields appropriately, but there are opportunities to enhance PII protection through encryption or masking, particularly for email addresses stored in groups and workflow parameters. The lack of explicit consent tracking for user PII is also a concern.

Third-Party & PII Flows — 4 finding(s). The layer shows potential risks related to PII handling when interacting with AI models and third-party services. There's a need for clearer consent mechanisms and data minimization strategies when processing personal data through AI-driven workflows.